To encourage experimentation and learning, we also deliberately omitted two features: the ability to edit existing knowledge bases and to review past chat sessions. Instead, we leave these as exercises for readers to implement using the codebase as a foundation.

11.1.2 Technical implementation

The web application uses the key components for building conversational UI web applications discussed in this book, with some additions.

Project code: rag

The code for this example project is in the ch11/rag folder. You can use that example by running this command in the root folder of the project repository:

```
$ npm run dev -w ch11/rag
```

You'll need the following API keys for this to work, as the application uses services from different vendors: the Gemini API key, the Upstash Redis and Vector REST URLs, the Upstash Redis and Vector REST Tokens, and the Clerk.js authentication credentials. These are configured as environment variables:

```
// .env
GEMINI_API_KEY=<GEMINI_API_SECRET_KEY>
UPSTASH_REDIS_REST_URL=<UPSTASH_REDIS_REST_URL>
UPSTASH_REDIS_REST_TOKEN=<UPSTASH_REDIS_REST_TOKEN>
UPSTASH_VECTOR_REST_URL=<UPSTASH_REDIS_REST_URL>
UPSTASH_VECTOR_REST_TOKEN=<UPSTASH_REDIS_REST_TOKEN>

NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=<CLERK_PUBLIC_KEY>
CLERK_SECRET_KEY=<CLERK_PRIVATE_KEY
```

Instructions on how to set up and create secret keys for Gemini AI and Upstash Redis are provided in the appendix.

11.1.3 Technology stack overview

Our RAG web application is built on a stack of technologies explored in this book, using the strengths of several key tools:

- *Next.js framework*—Next.js provides a versatile environment for building dynamic web applications, supporting both server-side rendering and static site generation.
- *Vercel AI SDK*—The Vercel AI SDK is utilized to manage AI and conversational state capabilities, including streaming responses and seamless integration with Next.js server components. This integration enhances our application's ability to handle complex AI-driven interactions efficiently.